Fundusze Europejskie dla Rozwoju Społecznego
Rzeczpospolita Polska
Dofinansowane przez Unię Europejską

SKRYPT DO LABORATORIUM

WPROWADZENIE DO PRZETWARZANIA OBRAZÓW

ĆWICZENIE 1

Metody reprezentacji obrazów

Projekt: „Inżynier 5.0 - kształcenie na potrzeby gospodarki”
współfinansowany przez Unię Europejską ze środków Europejskiego Funduszu Społecznego Plus w ramach programu Fundusze Europejskie dla Rozwoju Społecznego 2021-2027 **nr umowy o dofinansowanie FERS.01.05-IP.08-0285/23-00**

Laboratorium 1.

Metody reprezentacji obrazów

Przedmiot: Wstęp do przetwarzania obrazów
Poziom: 3. semestr studiów inżynierskich

Po wykonaniu ćwiczenia będziesz wiedzieć jak:

- opisać obraz cyfrowy jako tablicę wartości pikseli;
- odczytać podstawowe informacje o obrazie: rozmiar, liczbę kanałów i typ danych;
- wyświetlić obraz kolorowy oraz jego kanały R, G i B;
- porównać dwie metody konwersji RGB do skali szarości;
- zastosować pseudokolorowanie do wizualizacji obrazu jednokanałowego.

Dla każdej ważniejszej operacji najpierw analizujemy jej **zapis matematyczny** i wykonujemy **obliczenia na tablicach** NumPy, a następnie **poznajemy gotową funkcję** z wybranej biblioteki realizującą tę samą operację.

Przygotowanie

W ćwiczeniu wykorzystamy biblioteki `NumPy`, `Matplotlib` oraz `OpenCV` (`cv2`). OpenCV będzie służyć do wczytywania, konwersji i zapisywania obrazów, natomiast NumPy — do bezpośrednich operacji na tablicach pikseli. Podczas zajęć oprócz przykładów z instrukcji zostaną podane przez prowadzącego **zadania do rozwiązania**.
Odpowiedzi do zadań należy umieścić w sprawozdaniu i wysłać na e-nauczanie na zakończenie zajęć.

Jeśli potrzebujesz instalacji bibliotek, wykonaj w terminalu aktywnego środowiska Conda:

```
conda install -c conda-forge numpy matplotlib opencv
```

> **Ważne:** `cv2.imread()` wczytuje obraz kolorowy domyślnie w kolejności kanałów BGR. Przed wyświetleniem przez Matplotlib lub analizą jako RGB wykonujemy konwersję BGR → RGB. W dalszej części będziemy przechowywać obraz w kolejności RGB.

1. Przygotowanie środowiska

Najpierw zaimportujmy potrzebne biblioteki. Uruchom poniższą komórkę.

In [1]:
```
import numpy as np
import matplotlib.pyplot as plt
import cv2

print("NumPy:", np.__version__)
print("OpenCV:", cv2.__version__)
```

```
NumPy: 2.4.6
OpenCV: 5.0.0
```

2. Obraz jako tablica danych

Obraz rastrowy składa się z pikseli ułożonych w wiersze i kolumny. W obrazie w skali szarości każdy piksel ma jedną wartość intensywności. W obrazie RGB każdy piksel ma trzy wartości: R, G i B.

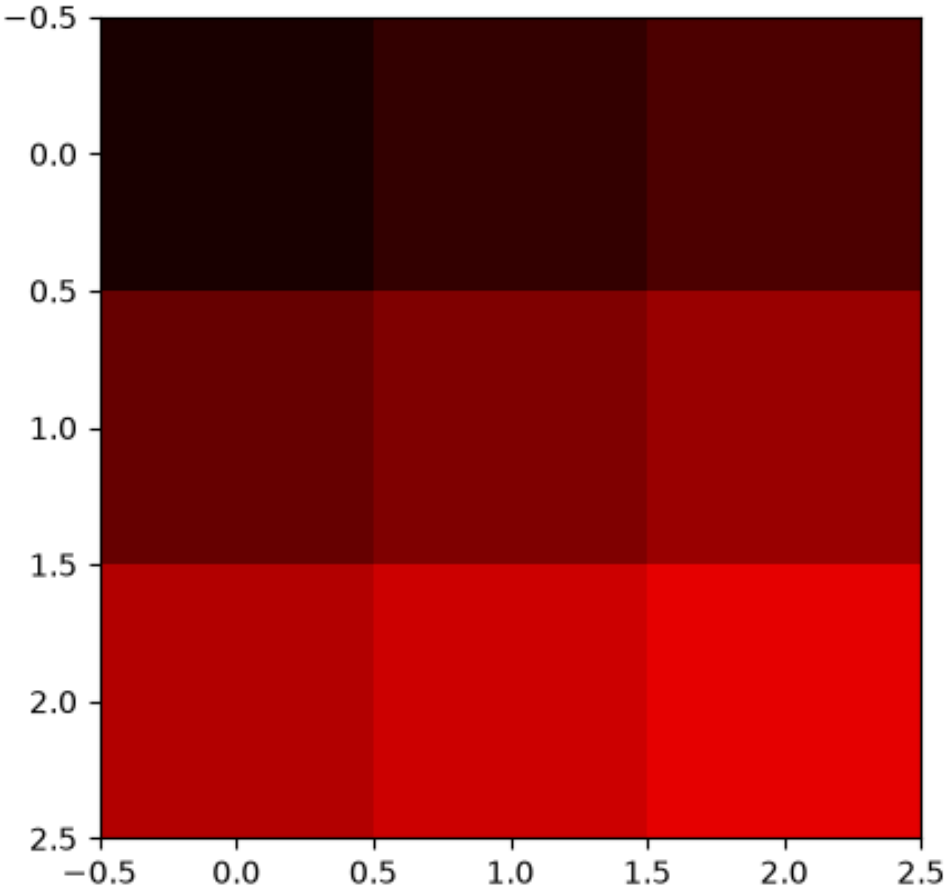
Zacznijmy od niewielkiego, sztucznego obrazu RGB. Każdy kanał ma rozmiar 3 × 3, a wartości w tym przykładzie należą do zakresu od 0 do 1. W tablicy NumPy element `C[i, j, k]` oznacza wartość kanału `k` piksela leżącego w wierszu `i` i kolumnie `j`. Dla obrazu RGB indeksy kanałów to: `0` — R, `1` — G, `2` — B.

In [2]:
```
# Utworzenie macierzy obrazu RGB o wymiarach 3 × 3 piksele
C = np.zeros((3, 3, 3), dtype=float)

# W kanale czerwonym umieszczamy różne intensywności
C[:, :, 0] = np.array([
    [0.1, 0.2, 0.3],
    [0.4, 0.5, 0.6],
    [0.7, 0.8, 0.9]
])

plt.imshow(C)
```

Out[2]:
```
<matplotlib.image.AxesImage at 0x20bd9afe090>
```



Wyświetliliśmy obraz za pomocą polecenia plt.imshow(C). Możemy jeszcze dopracować wyświetlanie naszej macierzy wykorzystując polecenia:

Polecenie	Działanie
`plt.imshow(C)`	Wyświetlamy macierz jako obraz, odwzorowując wartości pikseli na kolory.
`plt.figure(figsize=(4, 4))`	Możemy kontrolować rozmiar wyświetlanego wykresu i obrazu, dopasowując go do pozostałych elementów notebooka.
`vmin=0, vmax=1`	Ustalamy stały zakres wartości odwzorowywanych na kolory. Ułatwia to porównywanie obrazów i zapewnia spójną interpretację jasności pikseli.
`plt.title(...)`	Dodajemy tytuł informujący, co przedstawia obraz, np. jaką operację wykonano lub który kanał obrazu jest prezentowany.
`plt.axis("off")`	Usuwamy osie, podziałki i numery pikseli, dzięki czemu obraz jest bardziej czytelny.
`plt.show()`	Jawnie wyświetlamy przygotowany obraz lub wykres. Jest to szczególnie przydatne, gdy w jednej komórce tworzymy kilka wykresów.

In [3]:
```
plt.figure(figsize=(4, 4))
plt.imshow(C, vmin=0, vmax=1)
plt.title("Obraz RGB – niezerowe wartości tylko w kanale R")
plt.axis("off")
plt.show()

print("Kształt tablicy:", C.shape)
print("Typ danych:", C.dtype)
```



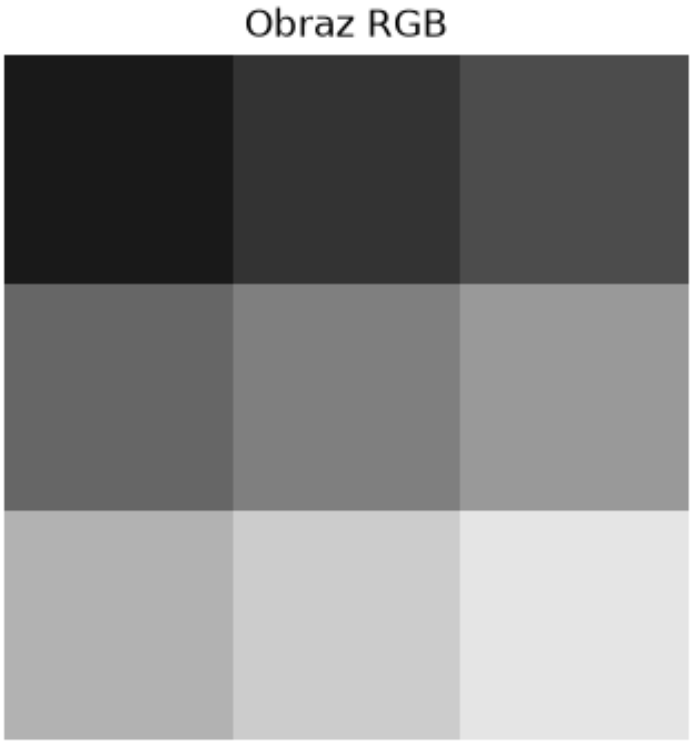

```
Kształt tablicy: (3, 3, 3)
Typ danych: float64
```

Utworzona tablica ma trzy wymiary: wysokość, szerokość i liczbę kanałów. W tym przykładzie kanały G i B mają same zera, dlatego obraz zawiera wyłącznie składową czerwoną.

Sprawdźmy, co się stanie, kiedy pozostałe kanały, G i B, uzupełnimy tymi samymi wartościami:

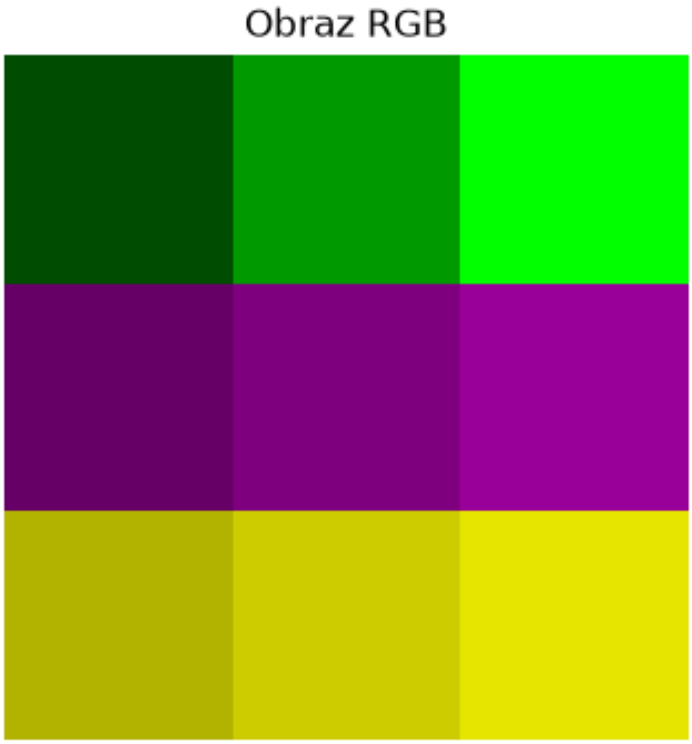
In [4]:
```
C[:, :, 1] = np.array([
    [0.1, 0.2, 0.3],
    [0.4, 0.5, 0.6],
    [0.7, 0.8, 0.9]
])
C[:, :, 2] = np.array([
    [0.1, 0.2, 0.3],
    [0.4, 0.5, 0.6],
    [0.7, 0.8, 0.9]
])
plt.figure(figsize=(4, 4))
plt.imshow(C, vmin=0, vmax=1)
```

```
plt.title("Obraz RGB")
plt.axis("off")
plt.show()
```



A teraz zmieńmy wartości tak, aby otrzymać różne kolory:

In [5]:
```
C[:, :, 0] = np.array([
    [0, 0, 0],
    [0.4, 0.5, 0.6],
    [0.7, 0.8, 0.9]
])
C[:, :, 1] = np.array([
    [0.3, 0.6, 1],
    [0, 0, 0],
    [0.7, 0.8, 0.9]
])
C[:, :, 2] = np.array([
    [0, 0, 0],
    [0.4, 0.5, 0.6],
    [0, 0, 0]
])
plt.figure(figsize=(4, 4))
plt.imshow(C, vmin=0, vmax=1)
plt.title("Obraz RGB")
plt.axis("off")
plt.show()
```



3. Wczytywanie obrazu z pliku

W kolejnych zadaniach wykorzystaj własny obraz kolorowy zapisany jako PNG lub JPEG. Umieść plik w podfolderze `dane` obok notebooka.

Przykładowa struktura katalogów:

```
laboratorium_01/
├── laboratorium_01.ipynb
└── dane/
    └── obraz_testowy.jpg
```

W komórce poniżej zmień nazwę pliku, jeśli korzystasz z innego obrazu.

In [6]:
```python
sciezka = "dane/flamingi.jpg"

# OpenCV wczytuje obraz kolorowy w kolejności BGR
obraz_bgr = cv2.imread(sciezka)

# Konwersja BGR -> RGB
obraz = cv2.cvtColor(obraz_bgr, cv2.COLOR_BGR2RGB)

print("Wymiary tablicy:", obraz.shape)
print("Typ danych:", obraz.dtype)
print("Minimalna wartość:", obraz.min())
print("Maksymalna wartość:", obraz.max())


plt.imshow(obraz)
plt.title("Obraz wejściowy")
```

```
plt.axis("off")
plt.show()
```

```
Wymiary tablicy: (225, 300, 3)
Typ danych: uint8
Minimalna wartość: 0
Maksymalna wartość: 255
```



- `obraz.shape` zwraca `(wysokość, szerokość, 3)` dla obrazu RGB.
- Typ `uint8` oznacza, że każda składowa piksela jest przechowywana jako 8-bitowa liczba całkowita bez znaku.
- W typowych obrazach RGB zapisanych w 8 bitach na kanał wartości składowych mieszczą się w zakresie 0–255.

Odczytywanie wartości pikseli i wyodrębnianie kanałów RGB

Obraz kolorowy jest przechowywany w Pythonie jako trójwymiarowa tablica NumPy. Dwa pierwsze wymiary określają położenie piksela (wiersz i kolumnę), a trzeci odpowiada za trzy składowe barwne: R, G i B. Dla przykładu obraz o rozdzielczości 480 × 640 pikseli ma wymiary tablicy (480, 640, 3).

1. Odczytywanie wartości pojedynczego kanału piksela
   Aby odczytać wartość jednego kanału wybranego piksela, należy podać jego położenie oraz numer kanału:

In [7]:
```
wiersz = 50
kolumna = 100
R = obraz[wiersz, kolumna, 0]
print("Wartość kanału R:", R)
```

```
Wartość kanału R: 44
```

2. Odczytywanie wszystkich wartości RGB jednocześnie
   Zamiast odczytywać każdy kanał osobno, możemy pobrać wszystkie trzy wartości za pomocą zapisu :

In [8]:
```
print("Wartości RGB:", obraz[wiersz, kolumna, :])
```

```
Wartości RGB: [44 44 32]
```

Znak : oznacza pobranie wszystkich elementów z danego wymiaru tablicy.

3. Wyodrębnianie i wyświetlanie poszczególnych kanałów
   Możemy również wyodrębnić z całego obrazu pojedyncze kanały barwne:

In [9]:
```
kanal_R = obraz[:, :, 0]
kanal_G = obraz[:, :, 1]
kanal_B = obraz[:, :, 2]
```

a następnie je wyświetlić:

In [10]:
```
plt.imshow(kanal_R, cmap="gray", vmin=0, vmax=255)
plt.title("kanal_R")
plt.axis("off")
plt.show()

plt.imshow(kanal_G, cmap="gray", vmin=0, vmax=255)
plt.title("kanal_G")
plt.axis("off")
plt.show()

plt.imshow(kanal_B, cmap="gray", vmin=0, vmax=255)
plt.title("kanal_B")
plt.axis("off")
plt.show()
```

kanal_R



kanal_G



kanal_B



Możemy też wyświetlić je inaczej.

Poniższy kod wyświetla wszystkie trzy kanały obok siebie w skali szarości:

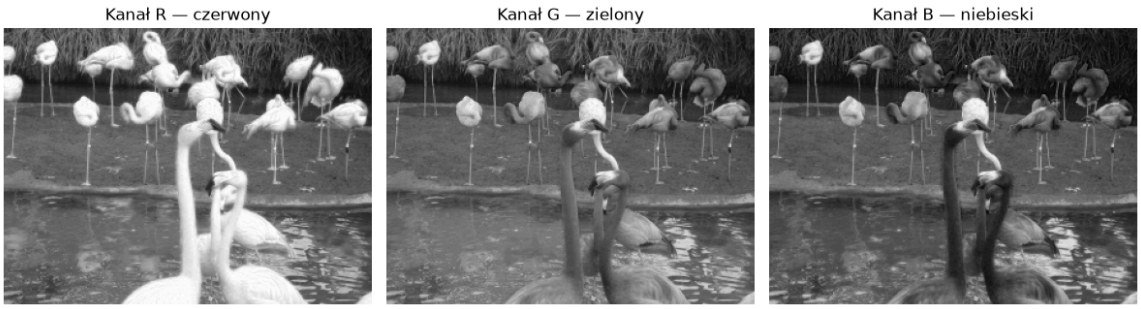
In [11]:
```
fig, osie = plt.subplots(1, 3, figsize=(12, 4))

osie[0].imshow(kanal_R, cmap="gray", vmin=0, vmax=255)
osie[0].set_title("Kanał R – czerwony")
osie[0].axis("off")

osie[1].imshow(kanal_G, cmap="gray", vmin=0, vmax=255)
osie[1].set_title("Kanał G – zielony")
osie[1].axis("off")

osie[2].imshow(kanal_B, cmap="gray", vmin=0, vmax=255)
osie[2].set_title("Kanał B – niebieski")
osie[2].axis("off")

plt.tight_layout()
plt.show()
```



4. Konwersja obrazu RGB do skali szarości

Obraz kolorowy można przekształcić do postaci jednokanałowej na różne sposoby.

Metoda A — średnia arytmetyczna

Dla każdego piksela obliczamy:

$$I_{gray1} = \frac{R + G + B}{3}$$

Metoda traktuje wszystkie trzy składowe jednakowo.

Metoda B — ważona suma składowych

Często stosuje się zależność:

$$I_{gray2} = 0,299R + 0,587G + 0,114B$$

Współczynniki występujące przy składowych RGB nie są jednakowe, ponieważ układ wzrokowy człowieka wykazuje różną wrażliwość na poszczególne składowe widma światła. Największy udział przypisano składowej zielonej (G), mniejszy czerwonej (R), a najmniejszy niebieskiej (B).

Zastosowanie takiego ważenia pozwala uzyskać obraz w skali szarości, którego postrzegana jasność lepiej odpowiada jasności obrazu oryginalnego niż w przypadku zastosowania zwykłej średniej arytmetycznej trzech kanałów.

Aby przekształcić nasz wcześniej wczytany obraz RGB do obrazu monochromatycznego wykorzystamy gotową funkcję z biblioteki OpenCV cv2.cvtColor:

In [12]:
```
obraz_gray_opencv = cv2.cvtColor(obraz, cv2.COLOR_RGB2GRAY)
plt.imshow(obraz_gray_opencv, cmap="gray", vmin=0, vmax=255)
plt.title("Obraz w skali szarości")
plt.axis("off")
plt.show()
```



5. Pseudokolorowanie obrazu jednokanałowego

Pseudokolorowanie polega na przypisaniu różnym wartościom obrazu jednokanałowego różnych kolorów. Nie dodaje ono nowych informacji do danych — zmienia sposób ich wizualizacji.
Nazwy wszystkich dostępnych map kolorów z biblioteki Matplotlib można wyświetlić za pomocą polecenia plt.colormaps()

Porównajmy kilka map kolorów dla tego samego obrazu w skali szarości.

In [13]:
```
mapy = ["gray", "viridis", "plasma", "turbo"]


fig, osie = plt.subplots(1, len(mapy), figsize=(17, 4))

for ax, mapa in zip(osie, mapy):
    ax.imshow(obraz_gray_opencv, cmap=mapa, vmin=0, vmax=255)
    ax.set_title(mapa)
    ax.axis("off")

plt.tight_layout()
plt.show()
```



Implementacja pseudokolorowania: tablica LUT i OpenCV

Pseudokolorowanie można opisać jako funkcję przypisującą każdej wartości intensywności trójkę (R, G, B). Jest to tablica odwzorowania (LUT/CLUT). Najpierw zbudujemy prostą tablicę za pomocą interpolacji NumPy, a następnie użyjemy gotowej funkcji OpenCV `cv2.applyColorMap()`.

In [14]:
```
# Własna tablica kolorów: punkty kontrolne w zakresie intensywności 0–
poziomy = np.array([0, 64, 128, 192, 255])
kolory = np.array([
    [0, 0, 0],       # czarny
    [0, 0, 255],     # czerwony (RGB)
    [255, 255, 0],   # żółty
    [0, 255, 0],     # zielony
    [255, 255, 255]  # biały
], dtype=np.float32)

wartosci = np.arange(256)
LUT = np.stack([
    np.interp(wartosci, poziomy, kolory[:, kanal])
    for kanal in range(3)
], axis=1).astype(np.uint8)

# Indeksowanie LUT wartościami obrazu
obraz_pseudokolor_numpy = LUT[np.clip(obraz_gray_opencv, 0, 255).astype(np.uint8)]

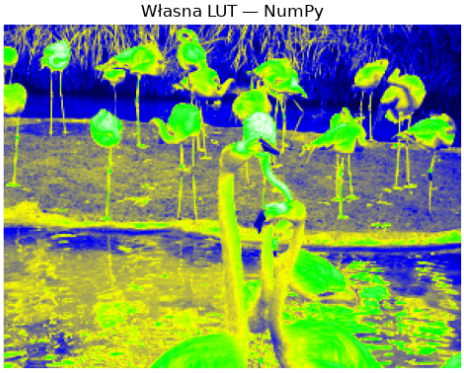

fig, osie = plt.subplots(1, 2, figsize=(15, 4))
osie[0].imshow(obraz_gray_opencv, cmap="gray", vmin=0, vmax=255)
osie[0].set_title("Dane wejściowe – szarość")
osie[1].imshow(obraz_pseudokolor_numpy)
osie[1].set_title("Własna LUT – NumPy")

for ax in osie:
    ax.axis("off")
plt.tight_layout()
plt.show()
```

Dane wejściowe — szarość



Własna LUT — NumPy



In [ ]: